Assignment 2

on

AGENTIC AI

School of Engineering and Technology

Department of Computer Science and Engineering

Subject code _ **CSCR 3215**

SUBMITTED BY:

Name:- Aishwarya Shelke
System ID:- 202300836

SUBMITTED TO:

Mr. Ayush Singh

Sharda University

Greater Noida, Uttar Pradesh

1. Introduction

This report documents the design, architecture, and implementation of an Autonomous Research Agent built for Assignment 2. The agent independently researches any user-provided topic by gathering information from multiple sources, synthesizing key insights, and generating a comprehensive structured report — all without human intervention beyond the initial topic input.

The system leverages LangChain's agent framework combined with Anthropic's Claude 3.5 Sonnet model, implementing the ReAct (Reasoning + Acting) pattern — which interleaves thought, action, and observation steps to adaptively explore a topic until sufficient information is gathered.

The agent is designed for researchers, students, analysts, and professionals who need rapid, well-organised overviews of complex topics without spending hours manually searching and synthesising sources.

2. Key Findings & Architecture

2.1 System Architecture

The agent follows a three-layer pipeline:

- Input Layer — accepts a free-text topic from the user via the command line
- Agent Core — ReAct loop powered by LangChain and Claude, orchestrating up to 12 reasoning iterations
- Output Layer — structured JSON research data converted into a formatted Markdown report

2.2 Tools Implemented

Three tools were integrated, exceeding the minimum requirement of two:

Tool	Provider	Purpose
Web Search	Tavily Search API	Real-time internet search — current statistics, news, case studies
Wikipedia	Wikipedia API	Background knowledge, definitions, historical context
LLM Analyzer	Anthropic Claude (custom tool)	Synthesises raw gathered text into structured key insights

2.3 ReAct Agent Loop

The agent implements the ReAct (Reasoning + Acting) pattern through an iterative cycle:

- Thought — the agent reasons about current state and what information is still needed
- Action — selects the most appropriate tool and formulates a targeted query
- Observation — receives and parses the tool's output
- Repeats up to 12 iterations, then produces a structured Final Answer in JSON

This design allows the agent to adaptively fill knowledge gaps, cross-reference sources, and build a comprehensive picture of any topic from multiple angles.

2.4 Report Generation

After the research loop completes, report_generator.py converts the structured JSON into a formatted Markdown report containing all required sections: Introduction, Key Findings, Challenges, Future Scope, Statistics, Conclusion, and Sources.

3. Challenges

3.1 Agent Output Parsing

The most significant technical challenge was reliably extracting structured JSON from the agent's free-form final answer. LLMs do not always produce clean JSON — they may wrap it in markdown code fences, add explanatory text, or produce partially malformed output. This was addressed with a two-layer parsing strategy:

- Primary: regex-based JSON extraction from the raw agent output
- Fallback: a second LLM call that explicitly prompts Claude to convert the raw output into clean JSON

3.2 Tool Output Variability

Web search results and Wikipedia extracts vary significantly in length, format, and relevance. The custom LLM Analyzer tool was added specifically to post-process raw tool outputs and extract only the most relevant insights, reducing noise before the agent synthesises its final answer.

3.3 API Rate Limits and Cost Control

Running a 12-iteration ReAct loop with multiple tool calls per iteration can consume significant API quota. The agent was tuned with temperature=0.3 to reduce token usage while maintaining factual accuracy. The max_iterations=12 cap prevents runaway agent loops while ensuring sufficient research depth.

3.4 Prompt Engineering for Structured Output

Getting the ReAct agent to reliably produce a JSON object with the required schema required careful prompt engineering. The system prompt explicitly defines the required JSON keys and instructs the agent to include a minimum number of findings, challenges, and future scope items — enforcing output quality at the prompt level.

4. Future Scope

4.1 Additional Data Sources

The current implementation uses Tavily (web) and Wikipedia (knowledge). Future versions could integrate:

- arXiv and PubMed for academic and scientific research
- SEC EDGAR and Bloomberg for financial and business topics
- PDF ingestion pipeline for uploading domain-specific documents
- YouTube transcript analysis for video-based knowledge

4.2 Multi-Agent Collaboration

The agent could be extended into a multi-agent system where specialised sub-agents handle distinct research tasks in parallel — one for web search, one for academic sources, one for data extraction — co-ordinated by an orchestrator agent. This would dramatically reduce research time for complex topics.

4.3 PDF and Presentation Output

Currently the agent produces Markdown reports. Adding LaTeX, docx, or PDF generation would make reports immediately shareable in professional contexts. Integration with Google Docs or Notion APIs would enable live collaborative research documents.

4.4 Interactive Web Interface

A Streamlit or FastAPI front-end would allow non-technical users to submit topics, monitor the agent's reasoning steps in real time (streaming the Thought/Action/Observation loop), and download reports without any command-line interaction.

4.5 Memory and Cross-Session Learning

Adding a vector database (e.g., ChromaDB or Pinecone) would allow the agent to store and retrieve past research, enabling cross-topic synthesis and avoiding redundant searches on related topics across sessions.

5. Sample Outputs

Two complete sample reports were generated to demonstrate the agent's capability across different domains:

#	Topic	Key Sections Covered
1	Impact of AI in Healthcare	Diagnostics, drug discovery, robotic surgery, mental health AI, NLP documentation — 8 cited sources
2	Role of Technology in Combating Climate Change	Renewable energy, EVs, green hydrogen, AI climate monitoring, carbon capture — 9 cited sources

6. Requirements Compliance

Requirement	Required	Status
Use LangChain framework	Yes	**Done**
Use Anthropic LLM (Claude)	Yes	**Done**
Web Search Tool	Yes	**Tavily**
Knowledge Tool (Wikipedia/PDF)	Yes	**Wikipedia**
Implement ReAct Agent	Yes	**Done**
Topic input from user	Yes	**CLI arg**
Search relevant information	Yes	**3 tools**
Extract key insights	Yes	**LLM Analyzer**
Organise content	Yes	**JSON schema**
Generate final report	Yes	**Markdown**
Cover Page in report	Yes	**Done**
Introduction section	Yes	**Done**
Key Findings section	Yes	**Done**
Challenges section	Yes	**Done**
Future Scope section	Yes	**Done**
Conclusion section	Yes	**Done**
2 Sample Outputs	Yes	**Done**
Source Code (GitHub-ready)	Yes	**Done**

7. Conclusion

The Autonomous Research Agent successfully fulfils all requirements of Assignment 2. By combining LangChain's ReAct agent framework with Anthropic Claude's advanced reasoning capabilities, three integrated tools, and a dedicated report generation pipeline, the system demonstrates end-to-end autonomous research from a single topic input to a structured, professional report.

The ReAct pattern proved highly effective for iterative research — the agent's ability to reason about gaps in its knowledge and adaptively select tools mirrors how a skilled human researcher would approach an unfamiliar topic. The two-layer JSON parsing strategy and LLM fallback ensure robust output regardless of the agent's exact response format.

The project is structured as a clean, modular Python codebase ready for GitHub submission, with comprehensive documentation, a requirements.txt, environment variable template, two complete sample outputs, and a clear README. Future extensions — multi-agent collaboration, additional data sources, a web UI — offer a clear roadmap for making this agent even more powerful.

8. Project File Structure

File	Description
agent.py	Main agent — LLM, tools, ReAct loop, and orchestration
report_generator.py	Converts research JSON into formatted Markdown report
requirements.txt	Python dependencies (pip install -r requirements.txt)
.env.example	API key template — copy to .env and fill in your keys
README.md	Full documentation, setup guide, usage examples
outputs/sample1_ai_healthcare_report.md	Sample Output 1: AI in Healthcare
outputs/sample2_climate_tech_report.md	Sample Output 2: Technology & Climate Change